B Aman Ulla Khan
https://github.com/Amaanafikhan/accuknox_assignment

Django Signals

Question 1: By default are Django signals executed synchronously or asynchronously?

Django signals are synchronous by default. I put a 5-second sleep in a post_save receiver and timed save() in the caller. It took about 5 seconds, so the caller was blocked until the receiver finished.

```python
@receiver(post_save, sender=Q1Record)
def q1_slow_receiver(sender, instance, created, **kwargs):
    print("[receiver] started, sleeping for 5 seconds...")
    time.sleep(5)
    print("[receiver] finished")
```

```python
    start = time.time()
    Q1Record.objects.create(name="q1 test")
    elapsed = time.time() - start
```

Question 2: Do Django signals run in the same thread as the caller?

Yes. I printed threading.get_ident() in both the caller and the receiver, and both showed the same thread ID (MainThread), so the receiver runs in the caller's thread.

```python
@receiver(post_save, sender=Q2Record)
def q2_thread_receiver(sender, instance, created, **kwargs):
    t = threading.current_thread()
    print(f"[receiver] thread id = {threading.get_ident()}, name = {t.name}")
```

```python
def handle(self, *args, **options):
    t = threading.current_thread()
    self.stdout.write(f"[caller]   thread id = {threading.get_ident()}, name = {t.name}")
```

Question 3: By default do Django signals run in the same database transaction as the caller?

Yes. Inside transaction.atomic(), I saved a record, the receiver wrote an AuditLog row, and then the caller raised an error on purpose. After the rollback, both rows were gone, so the receiver's write was part of the caller's transaction.

```python
@receiver(post_save, sender=Q3Record)
def q3_audit_receiver(sender, instance, created, **kwargs):
    in_atomic = transaction.get_connection().in_atomic_block
    print(f"[receiver] inside atomic block? {in_atomic}")
    AuditLog.objects.create(message=f"Q3Record '{instance.name}' saved")
```

```python
try:
    with transaction.atomic():
        Q3Record.objects.create(name="q3 test")
        self.stdout.write(
            f"[caller] inside the transaction: "
            f"Q3Record={Q3Record.objects.count()}, AuditLog={AuditLog.objects.count()}"
        )
        raise RuntimeError("forcing a rollback")
except RuntimeError:
    self.stdout.write("[caller] error raised -> transaction rolled back")
```

Custom Classes in Python

Rectangle class

The class takes length and width as integers. Its __iter__ method is a generator, so iterating over an instance yields {'length': value} first and then {'width': value}.

```python
class Rectangle:

    def __init__(self, length: int,width: int):
        self.length = length
        self.width = width


    def __iter__(self):
        yield {'length': self.length}
        yield {'width': self.width}
```

```python
class Command(BaseCommand):
    help = "Iterate over a Rectangle instance"

    def handle(self, *args, **options):
        rect = Rectangle(10, 5)
        for item in rect:
            self.stdout.write(str(item))
```